1. QR Codes

Access the Docker container, and then execute the tutorial commands within Docker to launch the application.

1.1. Introduction to QR Codes

A QR code is a type of two-dimensional barcode. "QR" stands for "Quick Response"—a name derived from the inventor's desire to create a code whose contents could be decoded rapidly. QR codes offer not only high data capacity, high reliability, and low cost, but they can also encode various types of textual information—including Chinese characters and images. Furthermore, they possess strong security and anti-counterfeiting capabilities, and are extremely convenient to use. Most importantly, the technology behind QR codes is open source.

1.2. Structure of a QR Code

Image	Description
	Positioning Markings: Indicate the orientation of the QR code.
	Alignment Markings: If the QR code is large, these additional elements assist in positioning.
	Timing Pattern: Through these lines, a scanner can determine the dimensions of the matrix.
	Version Information: Specifies the version number of the QR code being used; currently, there are 40 different QR code versions. Versions 1 through 7 are typically used within the retail sector.
	Format Information: The format pattern contains information regarding error correction levels and data masking patterns, making the code easier to scan.
	Data and Error Correction Keys: These patterns store the actual data.
	Quiet Zone: This area is crucial for scanners; its function is to isolate the QR code from its surrounding environment.

1.2.1: Characteristics of QR Codes

The data contained within a QR code includes redundant information. Consequently, even if up to 30% of the QR code's structure is damaged, its readability remains unaffected. A QR code offers a storage capacity of up to 7,089 bits or 4,296 characters; this includes punctuation marks and special characters, all of which can be encoded into the QR code. In addition to numbers and individual characters, entire words and phrases—such as URLs—can also be encoded. As more data is added to a QR code, the physical size of the code increases, and its structural complexity grows.

1.2.2: QR Code Generation and Recognition

Path to the provided source code package:

```
dofbot_pro_vision/dofbot_pro_vision/create_qrcode.py
```

After entering the Docker container, enter the following commands to install the required environment.

```
python3 -m pip install qrcode pyzbar
sudo apt-get install libzbar-dev
```

- Creation

Create a QRCode object.

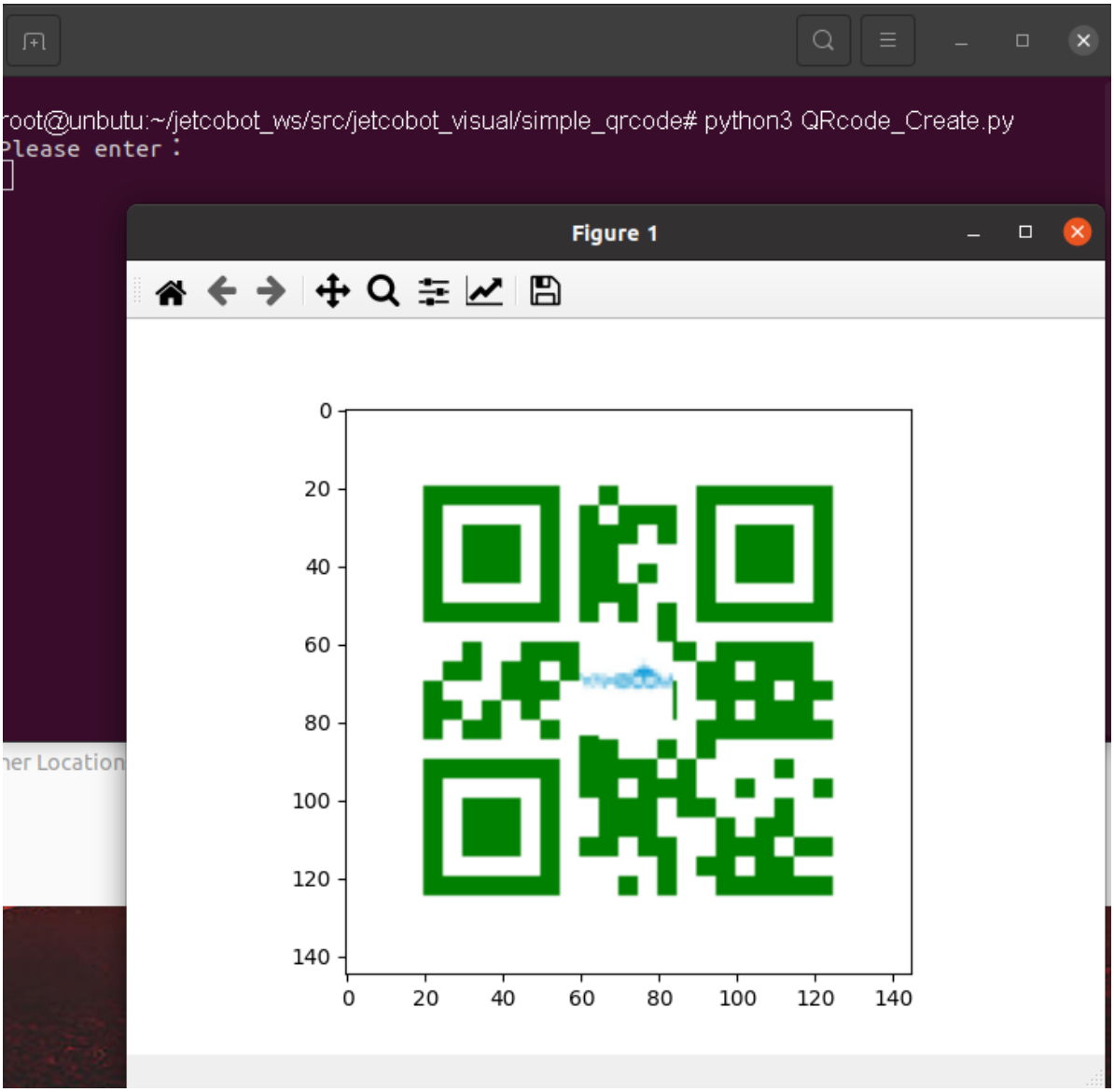
```
    '''
    Parameter Definitions:
    version: An integer value ranging from 1 to 40, controlling the size of the
QR     code (the minimum value is 1, which corresponds to a 12×12 matrix).
    If you wish for the program to determine this automatically, set the value to
    None and use the 'fit' parameter.
    error_correction: Controls the error correction capability of the QR code. It
    accepts one of the following four constants:
    ERROR_CORRECT_L: Approximately 7% or fewer errors can be corrected.
    ERROR_CORRECT_M (Default): Approximately 15% or fewer errors can be
corrected.
    ERROR_CORRECT_H: Approximately 30% or fewer errors can be corrected.
    box_size: Controls the number of pixels contained within each individual
"box"     (module) of the QR code.
    border: Controls the number of boxes (modules) constituting the border (the
      distance between the QR code and the image's edge). (The default is 4,
which is the minimum value specified by the relevant standards.)
    '''
    qr = qrcode.QRCode(
    version=1,
    error_correction=qrcode.constants.ERROR_CORRECT_H,
    box_size=10,
    border=4
)
```

Add a Logo to a QR Code

```
    # If the logo URL exists, add the logo image.
    my_file = Path(logo_path)
    if my_file.is_file(): img = add_logo(img, logo_path)
```

Note: When using Chinese, Chinese characters must be included.

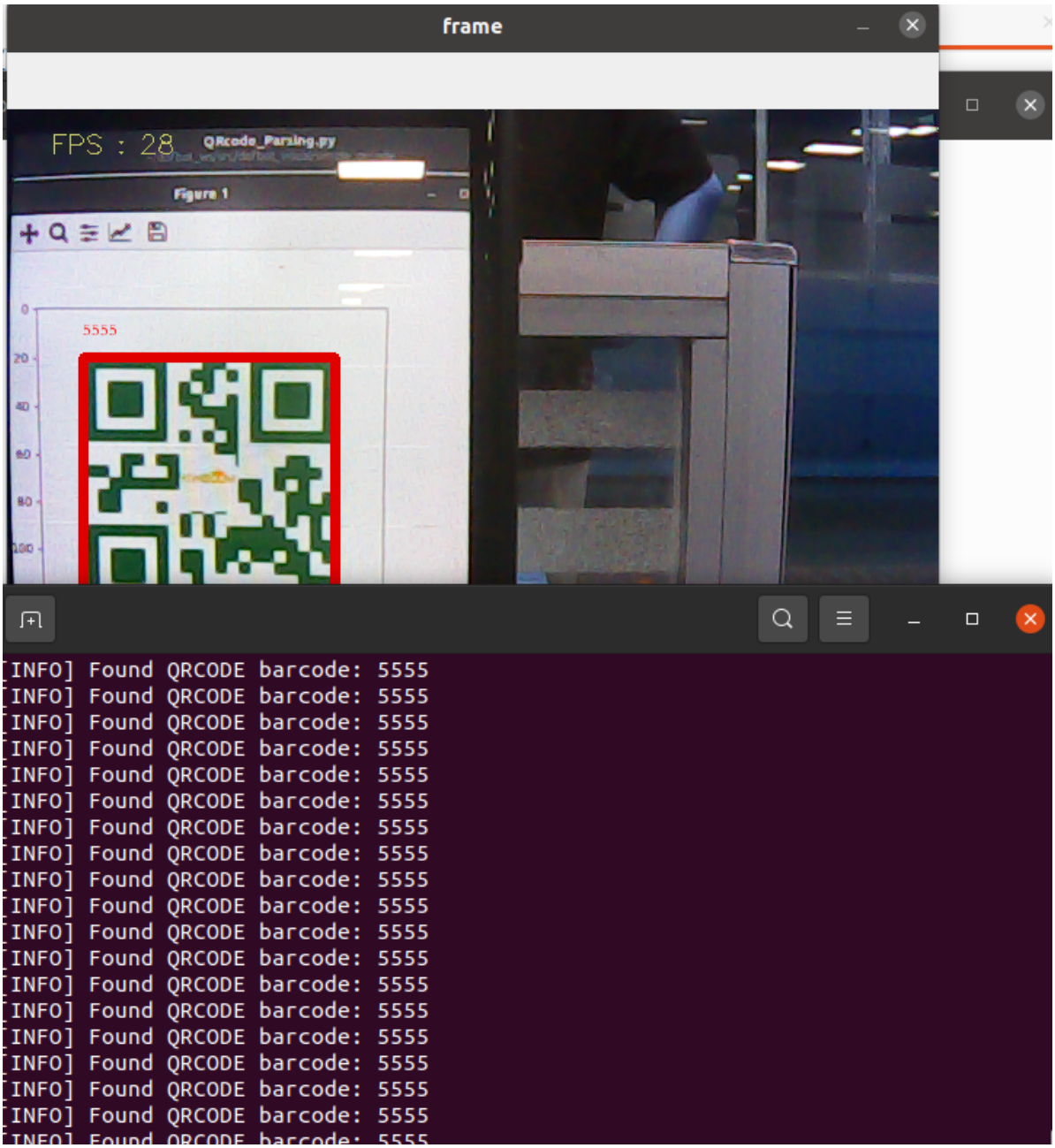
```
ros2 run dofbot_pro_vision create_qrcode
```



- Identification

```
def decodeDisplay(image, font_path):
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    # The output Chinese characters must first be converted into Unicode encoding form.
    barcodes = pyzbar.decode(gray)
    for barcode in barcodes:
        # Extract the bounding box position of the QR code
        (x, y, w, h) = barcode.rect
        # Draw bounding boxes around the barcodes in the image.
        cv.rectangle(image, (x, y), (x + w, y + h), (225, 0, 0), 5)
        encoding = 'UTF-8'
        # To render it, you first need to convert it into a string.
        barcodeData = barcode.data.decode(encoding)
        barcodeType = barcode.type
       # Plot Data and Types on the Image
        pilimg = Image.fromarray(image)
       # Create a Brush
        draw = ImageDraw.Draw(pilimg)
        # Parameter 1: Font file path, Parameter 2: Font size
        fontStyle = ImageFont.truetype(font_path, size=12, encoding=encoding)
       # Parameter 1: Print Coordinates, Parameter 2: Text, Parameter 3: Font Color, Parameter 4: Font
```

```
        draw.text((x, y - 25), str(barcode.data, encoding), fill=(255, 0, 0),
font=fontStyle)
        # Convert PIL Image to OpenCV Image
        image = cv.cvtColor(np.array(pilimg), cv.COLOR_RGB2BGR)
        # Print barcode data and barcode type to the terminal
        print("[INFO] Found {} barcode: {}".format(barcodeType, barcodeData))
    return image
```

- Demonstration

```
# Launch the camera (enter the Docker container terminal to execute):
ros2 launch dofbot_info camera_arm_kin.launch.py
# Launch the recognition program (enter the Docker container terminal to
execute):
ros2 run dofbot_pro_vision parse_qrcode
```